SKN35 · LLM 연동 내외부 문서 기반 질의응답 시스템

LangChain 버전 마이그레이션 코드 어시스턴트

옛 버전 코드를 원하는 버전에서 돌아가는 코드로, 근거와 검증까지 함께

SKN35 3rd 4Team · 발표 [이름] · 2026-10-15

목차

15분 동안 이 순서로 말씀드리겠습니다

1	문제와 해결	LLM은 옛 코드를 주고 틀려도 모른다 → 근거 + 검사
2	만든 것	데이터 · 그래프 · 검증 11가지 · 화면 · 대화 기록
3	평가와 결과	질문 30개 비교 설계 · 실제 실행 사례 · 결과
4	배운 것	발견 · 한계 · 가치

ChatGPT가 준 LangChain 코드, 실행하면 깨집니다

```
# LLM이 흔히 주는 코드

from langchain.chat_models import ChatOpenAI
from langchain.chains import LLMChain

---

# LangChain 1.4.4에서 실행 결과

ImportError: cannot import name 'ChatOpenAI'
ModuleNotFoundError: No module named 'langchain.chains'
```

첫 줄부터 에러

LangChain 1.0에서 패키지가 나뉘고, 옛 체인과 에이전트가 빠졌습니다.

더 큰 문제

LLM은 자기 답이 틀렸다는 것을 **모릅니다.**

인터넷에는 옛 버전 코드가 더 많습니다

LLM은 데이터가 많은 쪽을 따라갑니다. 라이브러리는 빨리 바뀌어도, 학습 데이터 속 코드는 대부분 과거 버전입니다.

42.55%

문서가 부족할 때, LLM 코드 중 목표 버전에서 실행되는 비율

When LLMs Lag Behind, 2026

66.36%

구조화된 문서를 줘도 실행되는 비율

When LLMs Lag Behind, 2026

48~51%

최신 상용 모델의 버전 지정 코드 생성 성공률

GitChameleon 2.0, 2025

→ 문서를 주는 것만으로는 부족하고, 고친 코드를 확인하는 단계가 필요합니다.

근거로 고치고, 검사로 한 번 더 확인합니다

구분	일반 LLM	우리 챗봇
근거	학습 데이터 (옛 버전이 다수)	import · 메서드 대응표 + 최신 공식 문서 검색 (RAG)
확인	없음	verifier가 LLM 없이 11가지 검사 (문법, import 존재, 남은 옛 import …)
틀렸을 때	모름	실패 이유로 근거를 다시 찾고, 틀린 근거는 빼고 다시 고침 (최대 3회)
답변	코드만	고친 코드 + 바뀐 줄마다 근거 ID · 이유 + 시도 횟수 + 설치할 패키지
버전	대부분 최신으로만	지금 버전 → 바꿀 버전 선택 (0.0.300 ~ 1.x)

고칠 때 볼 근거를 미리 만들어 두었습니다

9,155줄

import 대응표
옛 경로 → 1.x 경로

42개

메서드 대응표
.run() → .invoke() 등

295개

공식 문서
최신 271 · 가이드 24

11,945개

검색용 조각 (청크)
Chroma Cloud + BM25

출처 4가지

PyPI 패키지 코드	langchain-cli 변환 규칙	최신 공식 문서	0.3.30 옛 마이그레이션 가이드
버전별 도구 목록 23개	공식 자동 변환 규칙	임베딩: text-embedding-3-small	옛 방식 → 새 방식 설명

네 단계를 거치고, 실패하면 되돌아갑니다

질문 분류(migration · langchain · general) → 코드 변환 요청이면 아래 파이프라인 (LangGraph)



연결 · 재시도 판단 · 질문 분류: 1번 (LangGraph) · 변환 기준: LangChain 1.4.4 · 모델: gpt-4o-mini

verifier는 LLM 없이 11가지를 검사합니다

1 문법

파이썬 문법 통과

2 근거 ID

보여 준 근거만 썼는지

3 변경 기록

기록 없는 변경 = 환각

4 남은 옛 import

옛 경로가 남았는지

5 import 존재

1.x에 그 경로가 있는지

6 남은 옛 메서드

.run() 같은 옛 호출 (경고)

7 안 쓰는 import

필요 없는 import가 남았나

8 반환값 보존

문자열 결과를 잃지 않았나

9 정의 안 된 이름

실행하면 NameError

10 출력 꺼내기

['text'] 잘못 꺼내기

11 체인 입출력 키

실제 키와 같은지

오류 → 다시 시도
경고 → 결과에 표시

화면 · 4번

결과와 근거를 한 화면에서 봅니다

[화면 캡처]
확인용 예제를 보낸 뒤 결과 패널
(검증 통과 배지 · 팀 그래프 배지 · 변경표와 근거 ID)

대화형 + 세션 저장

ChatGPT처럼 대화 목록, 새 대화마다 세션 하나

버전 고르기

지금 코드 버전(자동 감지) → 바꿀 버전

결과 패널

검증 배지 · 전후 비교 · 근거 ID와 이유

안내

근거가 부족해 못 바꾼 줄 · 설치할 패키지

확인용 예제 30개

사이드바에서 골라 바로 보내기

TiDB에 사용자와 대화를 저장합니다

users

uuid	VARCHAR(36)	PK
user_id	VARCHAR(100)	UNIQUE
created_at	DATETIME	

1 : N

chat_history

uuid	VARCHAR(36)	PK
user_id	VARCHAR(100)	FK
session_id	VARCHAR(100)	IDX
question	TEXT	
answer	TEXT	
created_at	DATETIME	IDX

아이디 중복 검사

가입 전 확인 + UNIQUE

질문 + 답 = 한 행

새 대화 = 새 session_id

목록 · 다시 열기

session_id로 묶고 시간순으로

키가 없을 때

로컬 저장으로 넘어가고 이유 표시

같은 질문 30개, 같은 조건으로 비교합니다

1.x로 올리기 30개

10가지 유형 × 3: import 경로 · 로더 · 임베딩 · LLMChain → LCEL · 체인 조합 · RAG · 메모리 · 에이전트 · 출력 파서 · 실무 상황

난이도

쉬움 7 · 중간 15 · 어려움 8
실제 사용자처럼 '요청 문장 + 옛 코드'로 구성

미리 확인 (36/36)

옛 코드가 1.x에서 깨지는지, 기대 import가 되는지
+ 버전 오가기 6개 (되돌리기 · 중간 단계)

항목	규칙
비교 대상	일반 LLM vs 우리 챗봇
모델	양쪽 같은 [모델명], temperature 0
입력	같은 요청 문장 + 같은 코드
채점	verifier와 같은 검사 기준
최대 시도	3회 (우리 챗봇)
지표	검증 통과율 · 근거 포함률 · 재시도 효과

LLMChain 코드가 3회차에 통과했습니다

변경 전 (옛 코드)

```
from langchain.chat_models import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

llm = ChatOpenAI(temperature=0)
chain = LLMChain(llm=llm, prompt=PromptTemplate
    .from_template('{word}의 반대말은?'))
print(chain.run(word='크다'))
```

변경 후 (팀 그래프 결과)

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(temperature=0)
chain = PromptTemplate.from_template(
    '{word}의 반대말은?') | llm | StrOutputParser()
print(chain.invoke({'word': '크다'}))
```

3회

검사 통과까지 시도

6줄

바뀐 줄 모두 근거 ID
(imap · mmap · doc)

0줄

근거 부족으로 못 바꾼 줄

약 1분

한 번 변환에 걸린 시간

결과

[결과 한 줄 요약]

우리 챗봇 · 검증 통과율

[__]%

일반 LLM · 검증 통과율

[__]%

근거 포함률 [__]% · 질문 30개 기준

[그래프 삽입]
난이도 · 유형별 검증 통과율

다시 고치는 루프가 통과율을 [+__%p] 올렸습니다

[그래프 삽입]
1회차 → 2회차 → 3회차 누적 통과율

1회차 통과율

[__]%

최종 통과율 (최대 3회)

[__]%

통과까지 평균 시도

[__]회

만들면서 발견

통과했다고 끝이 아니었습니다

지원 종료 중인 패키지

langchain-community는 import만 해도 '지원 종료 중' 경고를 냅니다. 지금 통과해도 곧 깨질 수 있어요.

langchain-community 0.4.2 확인

classic 우회

LLMChain을 langchain_classic에서 가져오면 import는 통과하지만, 최신 방식(LCEL)은 아닙니다.

대응표 · 화면에 '옛 창고(classic)'로 표시

옛 버전끼리는 그대로

0.0.300 ~ 0.3.30 사이에는 자주 쓰는 import 경로가 거의 같습니다. 크게 바뀌는 것은 1.x와의 사이입니다.

버전별 도구 목록 23개 비교

통과가 곧 정답은 아닙니다

한계	다음 단계
코드 검사까지 하지만, 실제 실행 결과가 맞는지는 확인 못 함	가짜 LLM을 넣어 코드 전체를 실제로 실행
다시 쓰기는 1.x로 올리기 중심 (되돌리기는 화면 쪽 임시 처리)	목표 버전을 그래프 전체에 넘겨 모든 방향 지원
검사 · 재시도로 한 번 변환에 약 1분	진행 단계 표시, 쉬운 import는 대응표만으로 바로 답하기
질문 30개, 팀이 직접 구축 · LangChain만 대상	실사용 코드로 확장, pydantic · OpenAI SDK 등으로 확장

라이브러리가 바뀔 때마다, 근거와 함께 확인된 코드

시간 절감

에러를 하나씩 검색하는 대신, 근거가 붙은 답을 바로 확인

신뢰

바뀐 줄마다 근거 ID와 이유, 몇 번 만에 검사를 통과했는지 함께 제시

확장

대응표 · 문서 · 검사 기준만 바꾸면 다른 라이브러리에도 적용

팀

1번 · [이름]

그래프 · 백엔드

LangGraph 연결 · 질문 분류
analyzer · 재시도 판단

2번 · [이름]

데이터 · 검색

대응표 · 청킹 · Chroma
하이브리드 검색 · retriever

3번 · [이름]

변환 · 검증

State · rewriter
verifier 11가지 검사

4번 · [이름]

화면 · DB · 평가 · 발표

Streamlit 화면 · TiDB
질문 36개 · 보고서

감사합니다

질문 받겠습니다

팀 저장소 github.com/SKNETWORKS-FAMILY-AICAMP/SKN35-3rd-4Team
배포 github.com/SKN35shimsungwook/langchain-migration-assistant · [배포 주소]